Design and Simulation of a 4-bit Arithmetic Logic Unit (ALU) using Verilog:

PROJECT DESCRIPTION:

This project focuses on the design and simulation of a **4-bit Arithmetic Logic Unit (ALU)** using the Verilog Hardware Description Language. The ALU is a fundamental component of digital systems and processors, responsible for performing arithmetic and logical operations.

The designed ALU takes two 4-bit input operands (**A** and **B**) and a 2-bit select signal (**sel**) to determine the operation to be performed. Based on the select input, the ALU performs one of the following operations:

- Addition
- Subtraction
- Bitwise AND
- Bitwise OR

The functionality of the ALU is implemented using a combinational logic approach with a `case` statement inside an `always` block.

A separate testbench is developed to verify the correctness of the design by applying multiple test cases with different input combinations. The simulation results are analyzed using waveform outputs generated in a VCD (Value Change Dump) file and visualized using GTKWave.

ALU CODE:

```
module alu (
    input [3:0] A,
    input [3:0] B,
    input [1:0] sel,
    output reg [3:0] result
);

always @(*) begin
    case(sel)
        2'b00: result = A + B;  // ADD
        2'b01: result = A - B;  // SUB
        2'b10: result = A & B;  // AND
        2'b11: result = A | B;  // OR
```

```
        default: result = 4'b0000;
    endcase
end

endmodule
```

ALU TEST BENCH CODE:

```
`timescale 1ns/1ps

module alu_tb;

    reg [3:0] A;
    reg [3:0] B;
    reg [1:0] sel;
    wire [3:0] result;

    // Instantiate ALU
    alu uut (
        .A(A),
        .B(B),
        .sel(sel),
        .result(result)
    );

    initial begin
        // GTKWave dump setup
        $dumpfile("alu.vcd");      // file name
        $dumpvars(0, alu_tb);      // dump all variables

        // Initial values
        A = 0; B = 0; sel = 0;

        // Test Case 1
        #10 A = 4'd5; B = 4'd3;

        #10 sel = 2'b00; // ADD
        #10 sel = 2'b01; // SUB
        #10 sel = 2'b10; // AND
        #10 sel = 2'b11; // OR
```

```
        // Test Case 2
        #10 A = 4'd10; B = 4'd2;

        #10 sel = 2'b00;
        #10 sel = 2'b01;
        #10 sel = 2'b10;
        #10 sel = 2'b11;

        // Test Case 3 (edge)
        #10 A = 4'd15; B = 4'd1;

        #10 sel = 2'b00;
        #10 sel = 2'b01;
        #10 sel = 2'b10;
        #10 sel = 2'b11;

        #20 $finish;
    end

endmodule
```